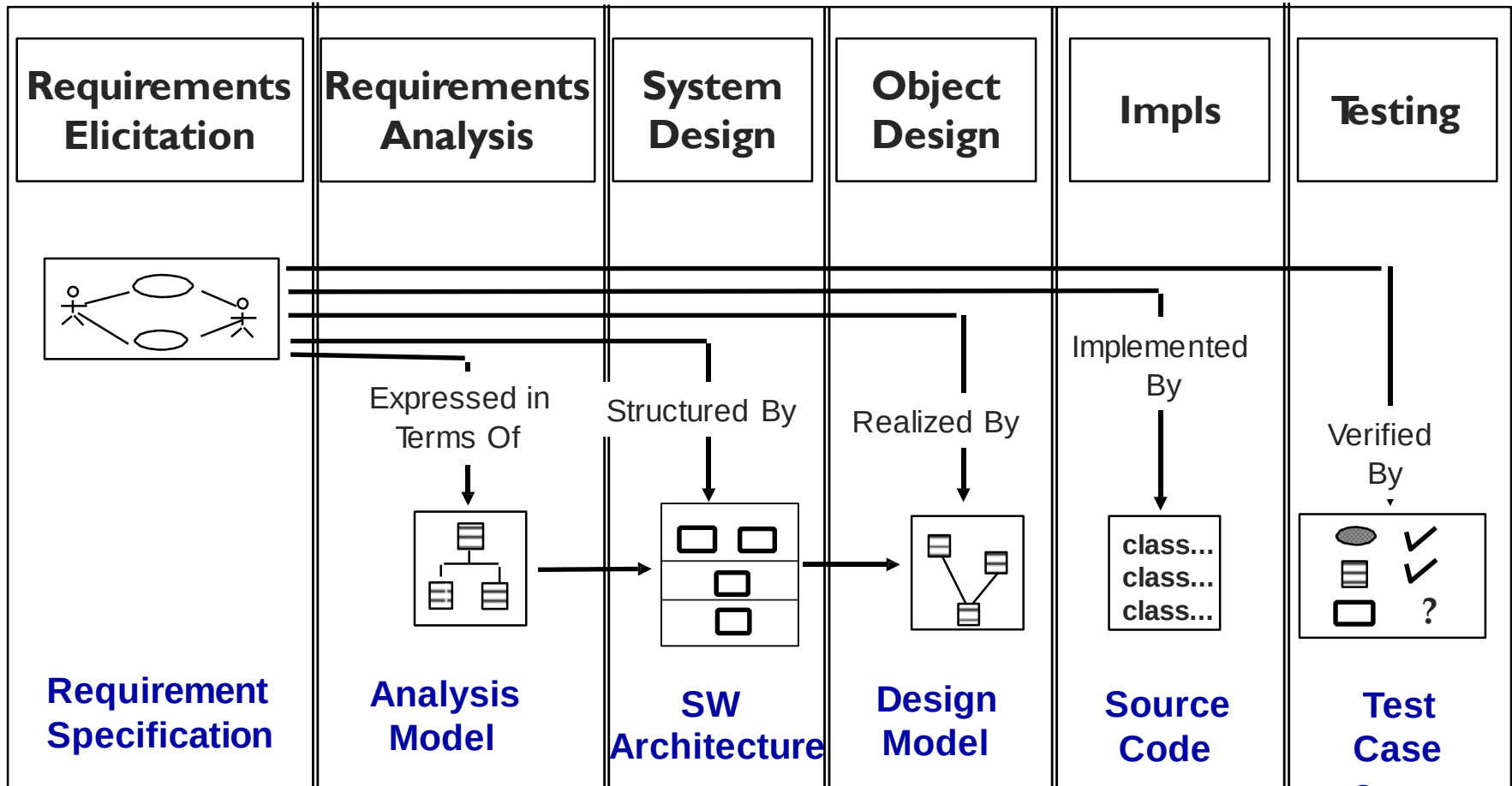


# CZ2006/CE2006 Software Engineering

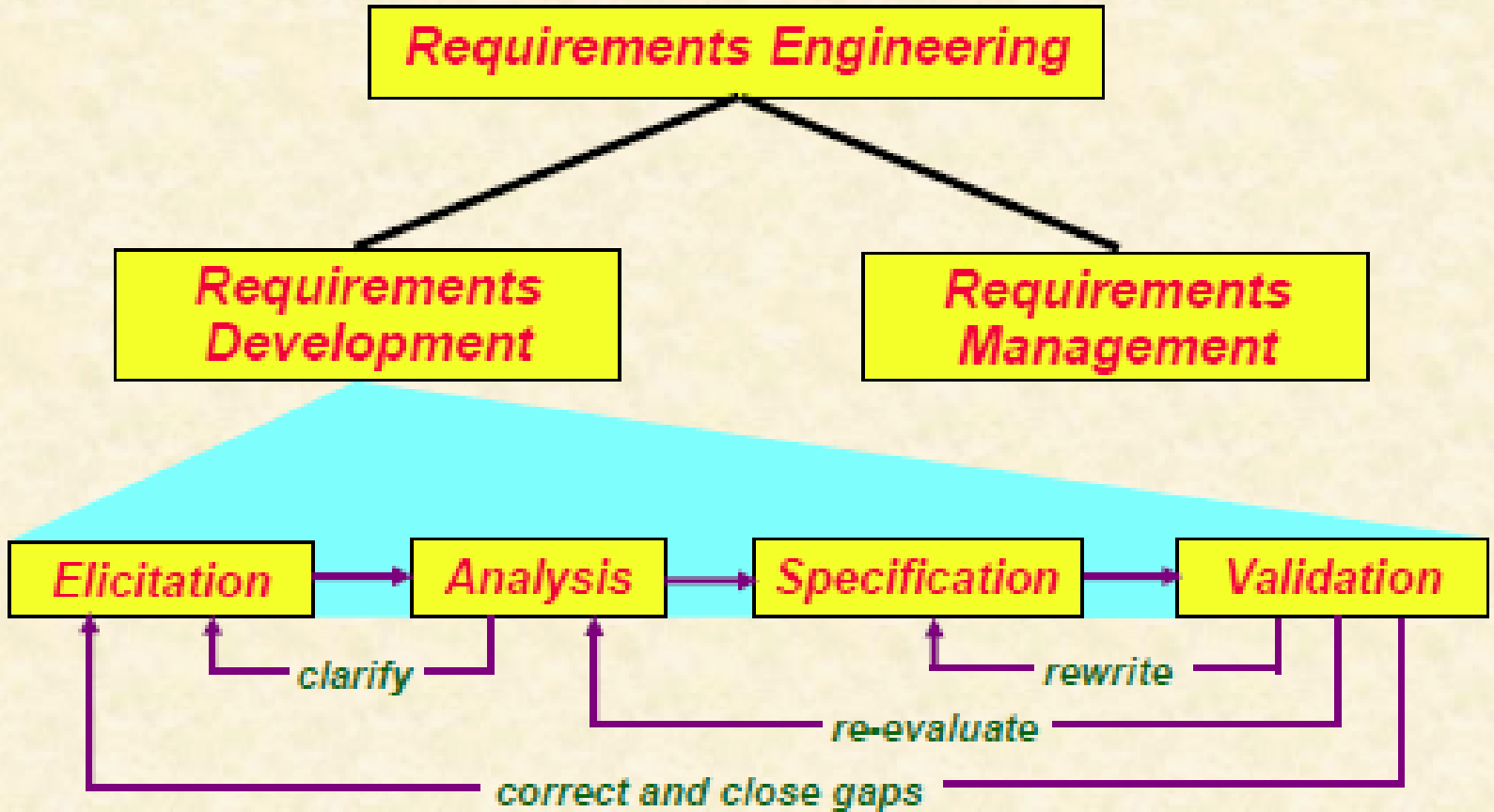
## Lecture 12: System Design

Liu Yang

# SDLC Activities

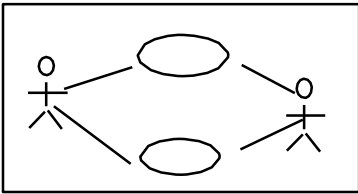


# Components of Requirements Engineering



# Understand What Customers Want

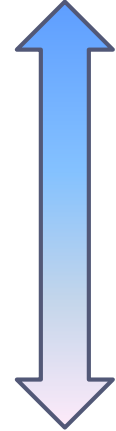
## Requirements Elicitation



Requirement  
Specification  
(SRS)

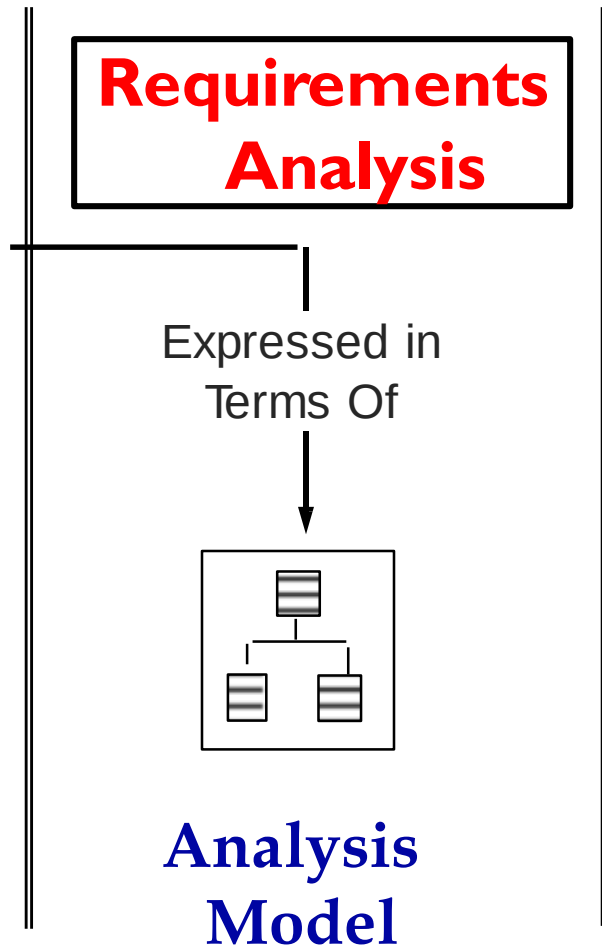
- ▶ From project mission statement
- ▶ To Requirement specification (SRS)
  - ▶ Func & non-func requirements
  - ▶ Data dictionary
  - ▶ Use case model

Informal  
Vague  
Incomplete



Formal  
Technical  
Consistent

# Bridge Customer Perspective to Software Engineer Perspective



## ► From Requirement Specification (SRS)

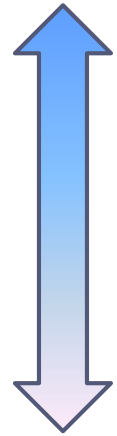
- Func & non-func requirements
- Data dictionary
- Use case model



## ► To analysis model

- Class diagram (boundary, control, entity)
- Sequence diagram
- State machine

**More  
about  
Customer**

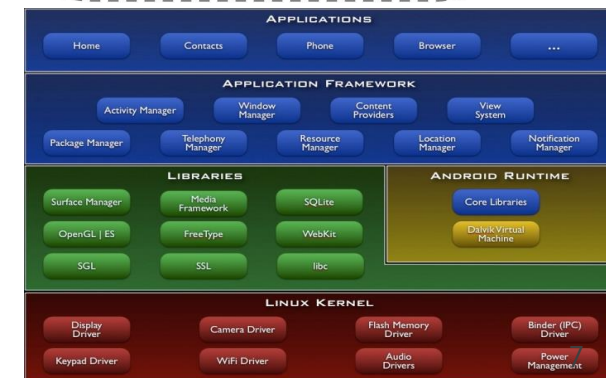
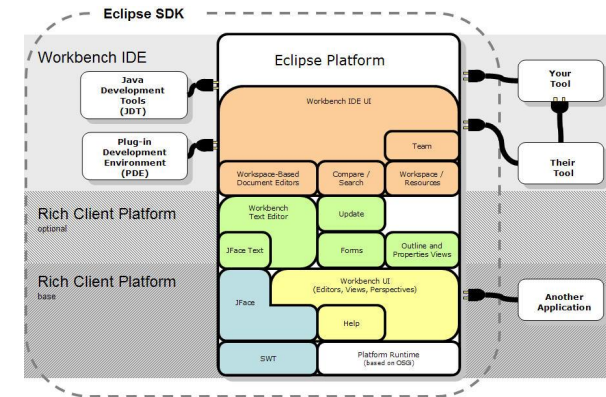
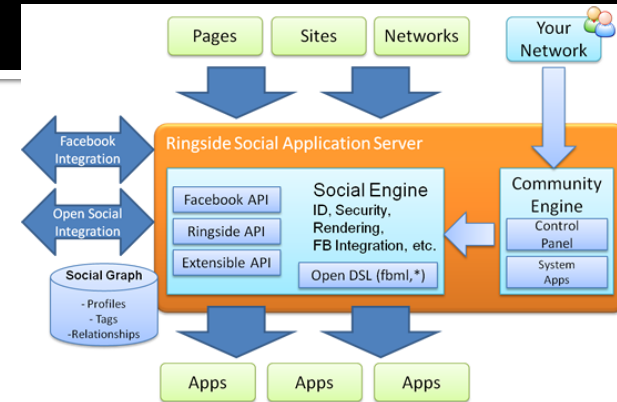


**More  
about  
Software  
Engineer**

# System Architecture Design, Why?

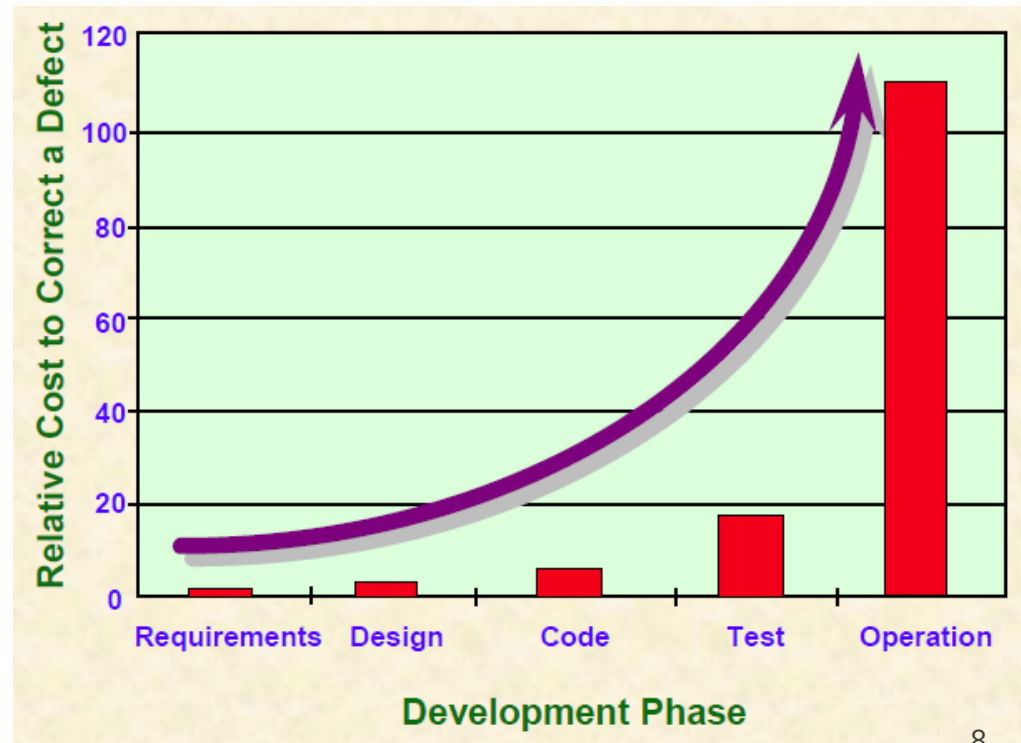
# The Reasons 1

- Larger
  - Source size, Binary size
  - Number of users, user input
  - Number of developers
  - Life of the project (decades...)
  - Number of changes, versions
- Complexity is increasing
  - Environment
  - Technologies
  - Hardware
  - ...



# The Reasons 2

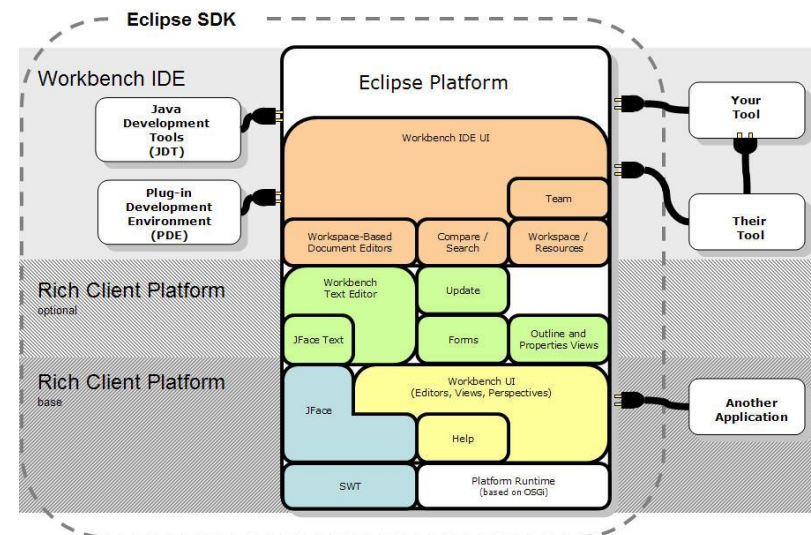
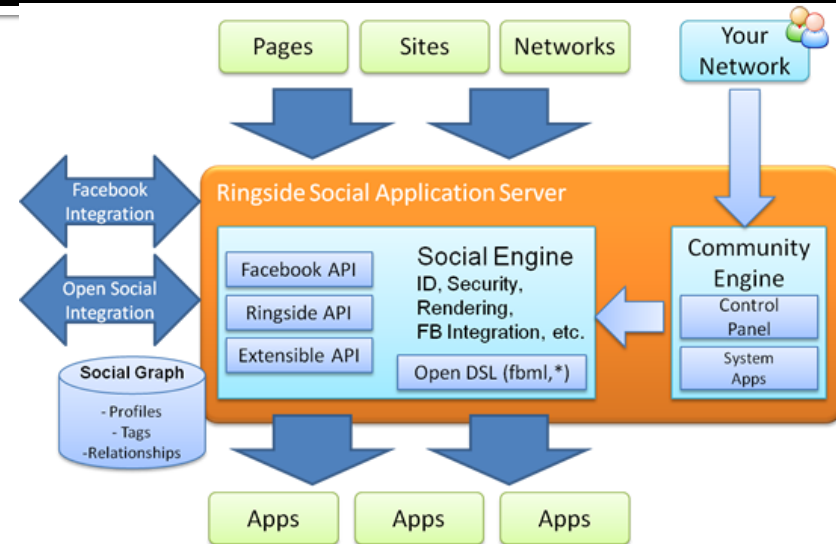
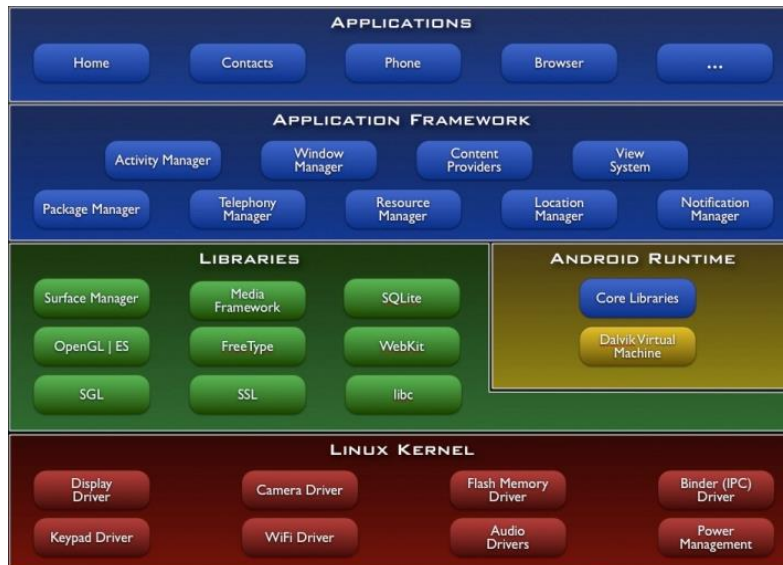
- Cost is critical (limited budget/resource)
- Schedule constraints (tight schedule)
- Maintenance





# The Reasons 3

- Non-functional Requirements
  - Performance
  - Availability
  - Security
  - Scalability
  - Extensibility



# Software System Architecture Design



# Software Architecture Definition

Software architecture is considered as a description of the high level structure of a software system in terms of architectural elements and the interactions between them.

*'Abstractly, software architecture involves the description of **elements** from which systems are built, **interactions** among those elements, **patterns** that guide their **composition**, and **constraints** on these patterns.'*

*(Shaw and Garlan, 1996)*

# Shaw and Garlan's Definition

*Software Architecture = {Components, Connectors}*

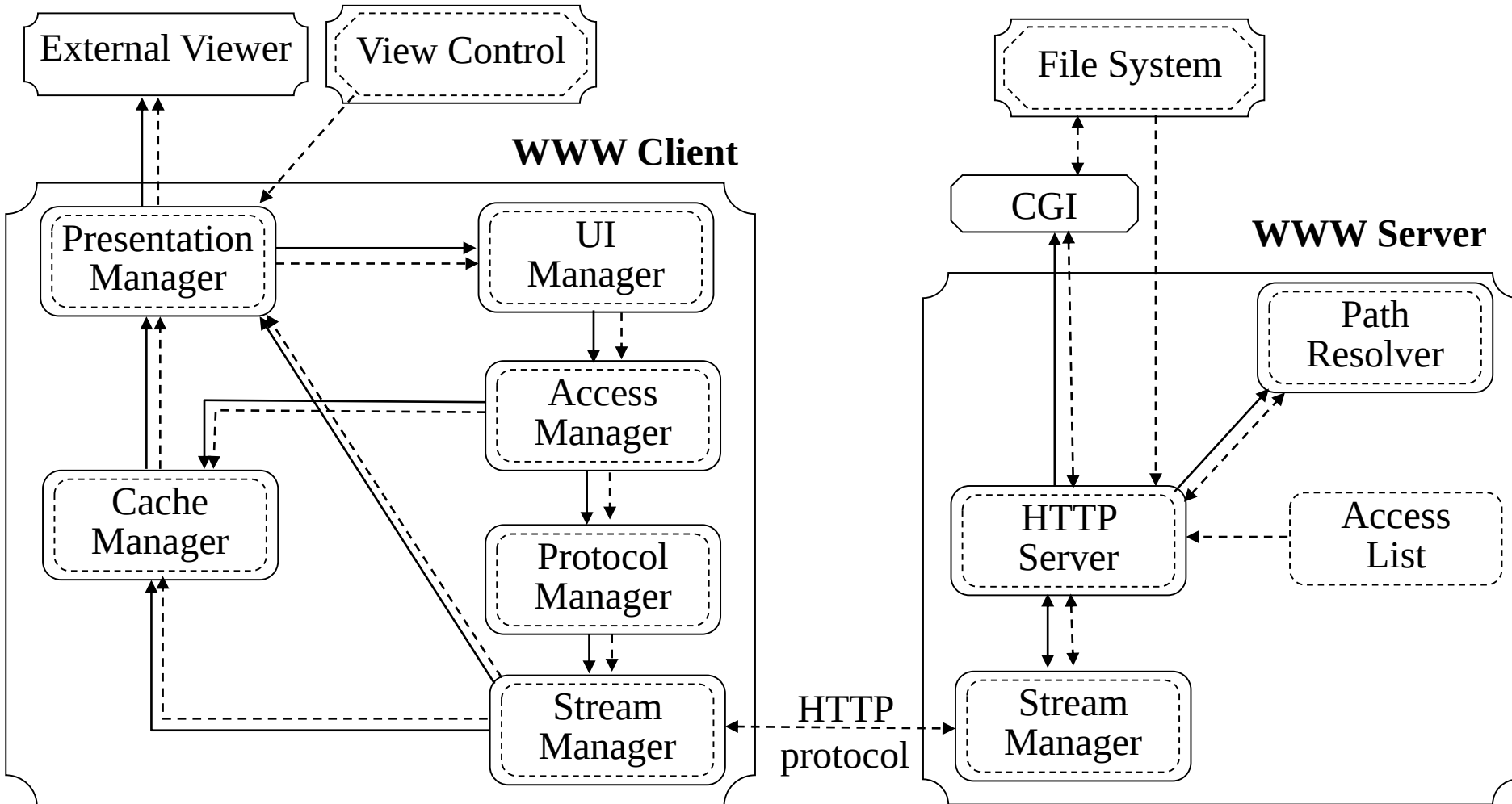
## ■ **Components**

- A component is a unit of software that performs some function at run-time.
- *Examples:* programs, objects, processes, clients, servers, databases

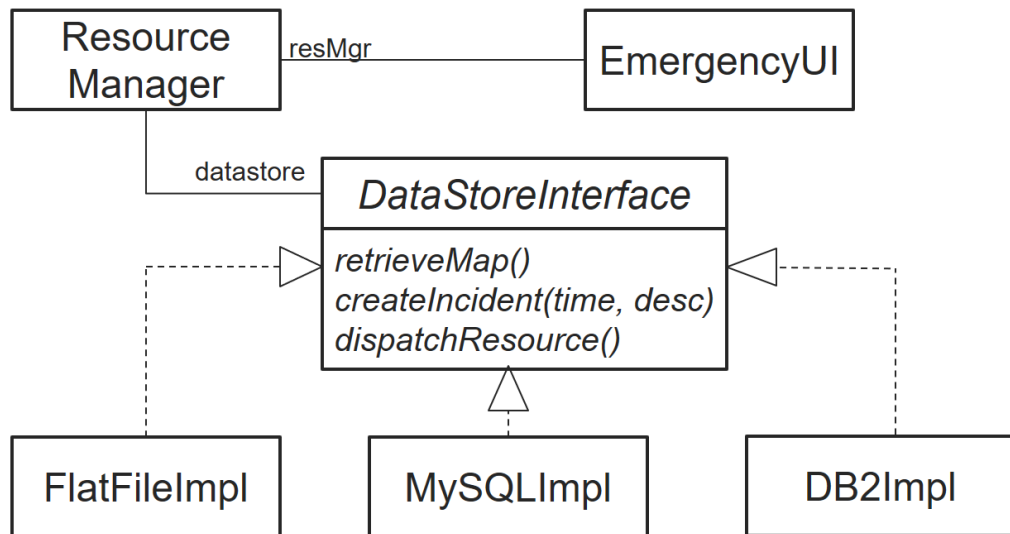
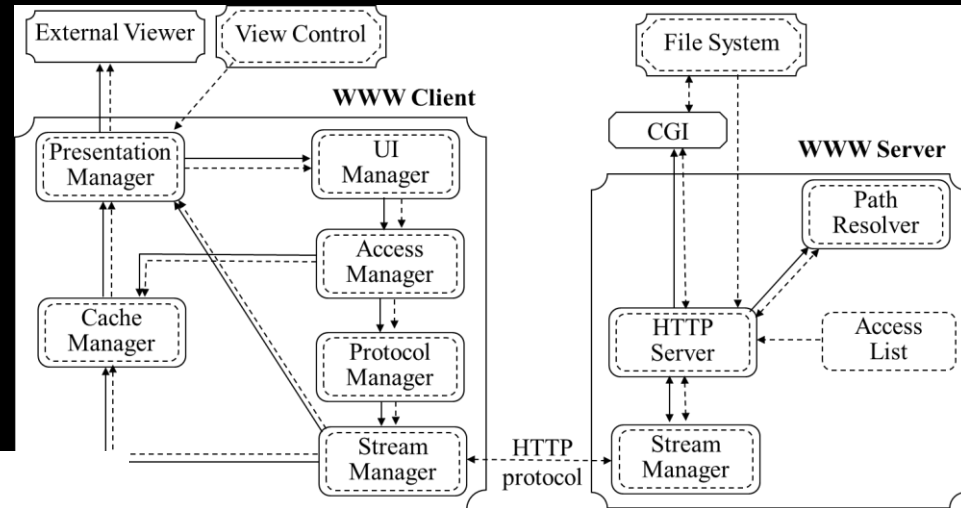
## ■ **Connectors**

- Interactions among components:
  - communication, coordination, or cooperation among components.
- *Examples:* shared variable access, procedure calls, remote procedure calls, communication protocols, data streams, transaction streams.
- Implementations of connectors are usually distributed over many system components; often do not correspond to discrete elements of the running systems.

# WWW Client-Server Architecture



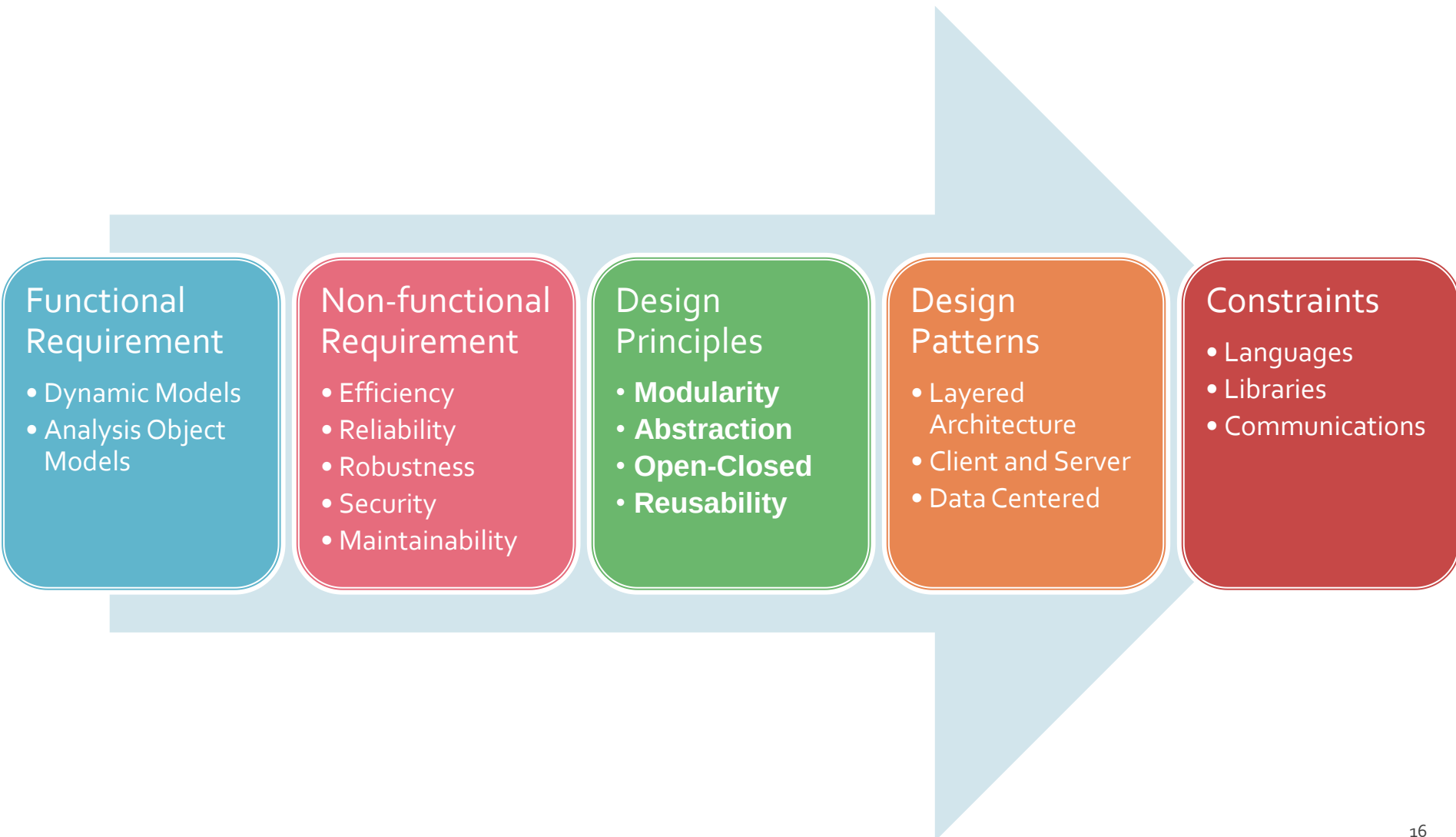
# Differences between class diagram and architecture diagram?



# Notes about SA

- Architecture is abstract
  - The details of the components and connectors are hidden
- Architecture is about structure and interactions
  - Focus on the topology
  - Components
  - Data and control communication
- Architecture is purposeful
  - demonstrate or analyse the properties of interest
  - design documentation, as transferable knowledge about software design, evaluation, and so on

# How to develop a system architecture?





*Don't repeat yourself*

*Modularity*

**Separation of Concerns**

**Low Coupling and High Cohesion**

*Open Close Principle*

*Principle of Least Knowledge*

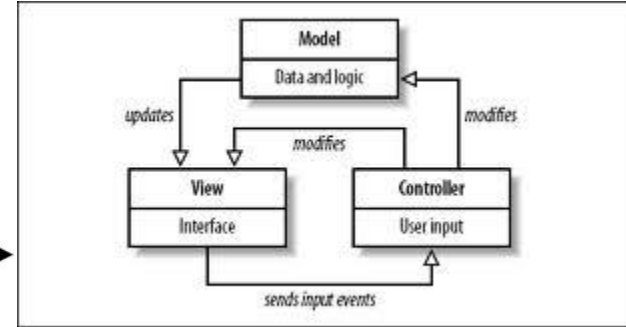
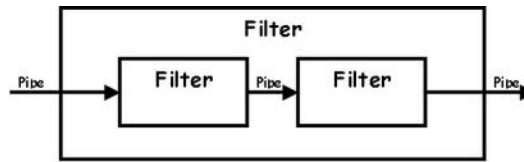
*Single Responsibility  
Principle*

# Software Architectural Styles

- One of the hallmarks of software architectural design is the use of idiomatic patterns of system organization.
- An architectural style defines a family of systems in terms of a **pattern** of **structural organization**.
  - A vocabulary of component and connector types
  - A set of constraints on how they can be combined
  - One or more semantic models that specify how to determine a system's overall properties from the properties of its parts

# What is in a Style

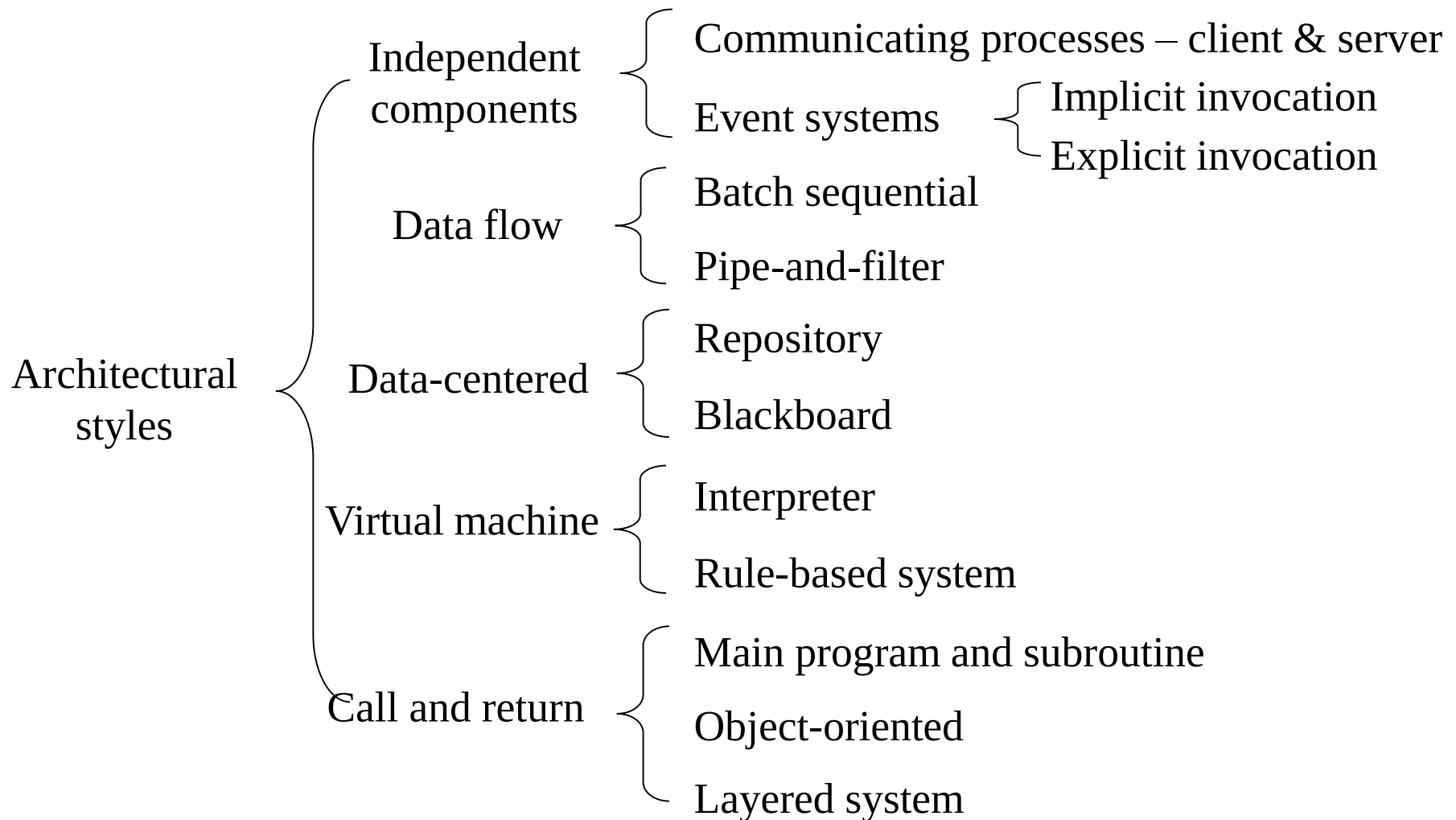
- A set of **component types**
  - Filters in pipe-and-filter styles
  - MVC
- A set of connectors
  - subroutine calls, remote procedure calls, data streams, and sockets
- A topological structure
- A set of semantic constraints
  - Filters in a pipe-and-filter architecture do not share states with each other
  - One layer can only call the service provided by the lower layer in a layered system



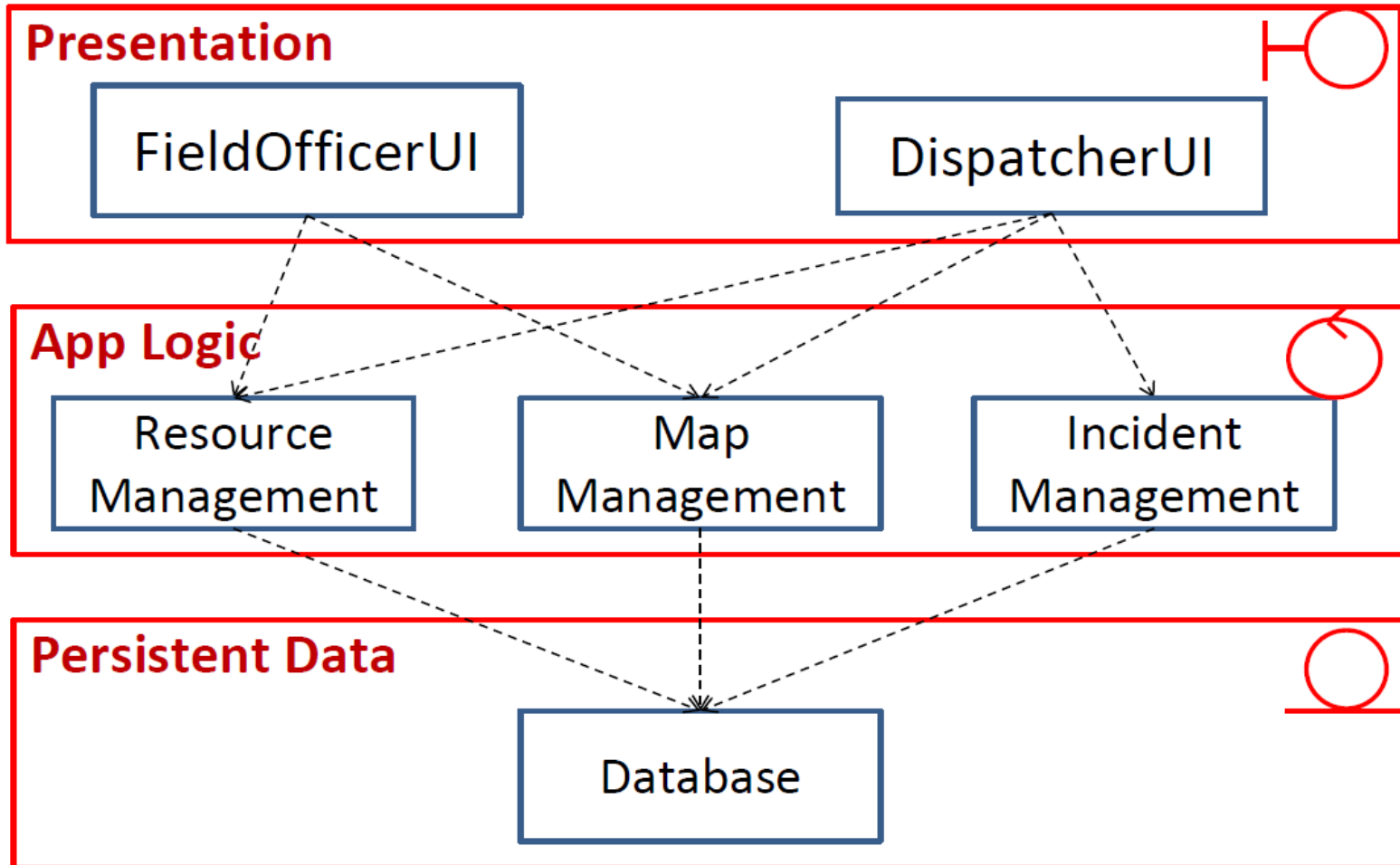
# What Is Ambiguous in A Style

- The number of components involved
  - a pipe-and-filter architecture
    - 2 filters connected by 1 pipe or
    - 20 filters connected by 19 pipes
- The mechanism of interaction
  - In a layered system, the call to the lower layer can be
    - local procedure calls, or remote procedure calls, or other process communication mechanisms.
    -
- The function of the system and components
  - one of the components in an architectural style may be a database, but the kind of data may vary.

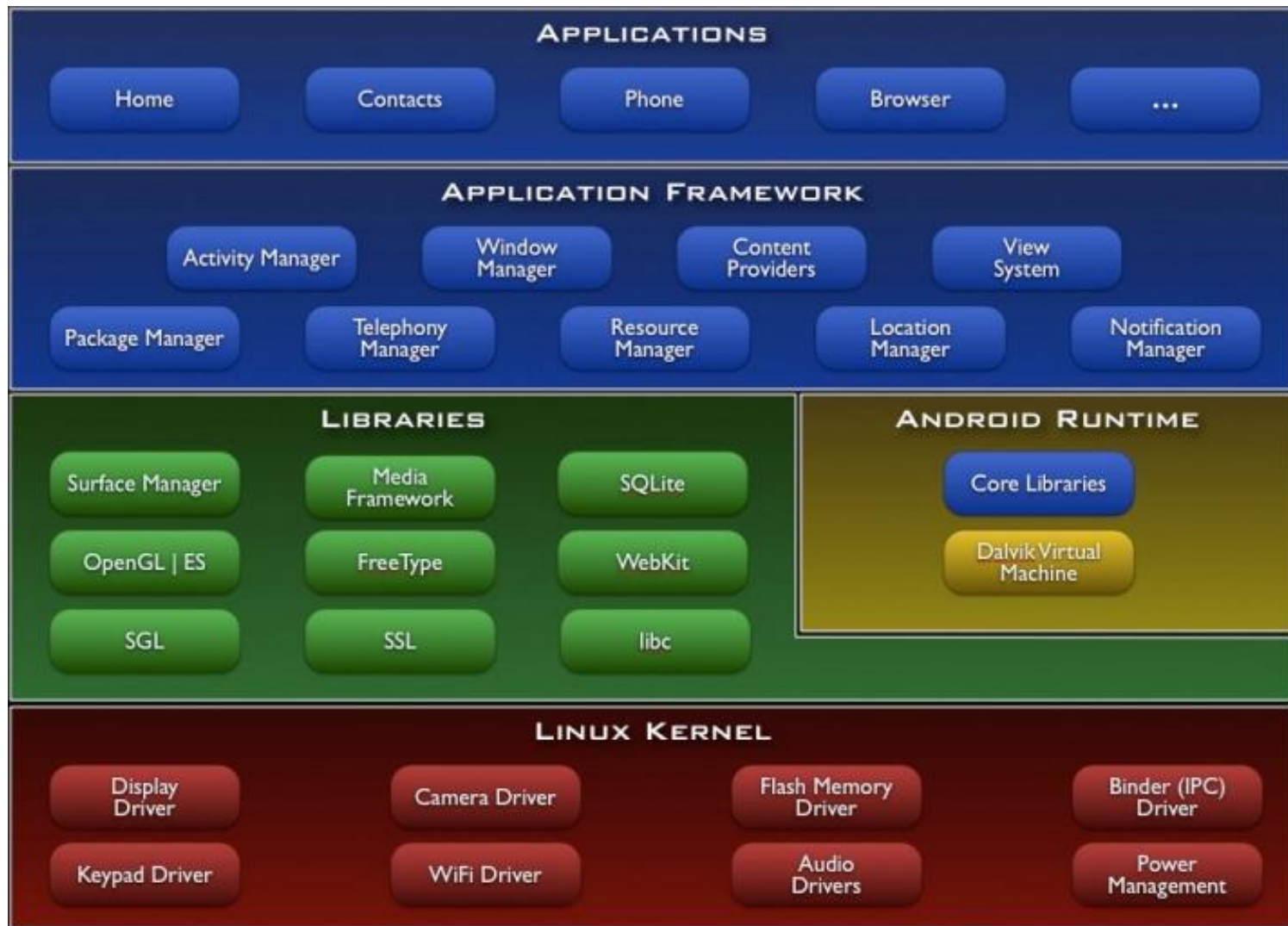
# A Catalogue of Software Architectural Styles



# Layered Architecture – 3-Layered Architecture



# Examples: Android



# Analysis of Independent Component

## Advantages

- Modifiability by decoupling the computation
- Concurrent execution
- Scalability
- Easy integration

## Disadvantages

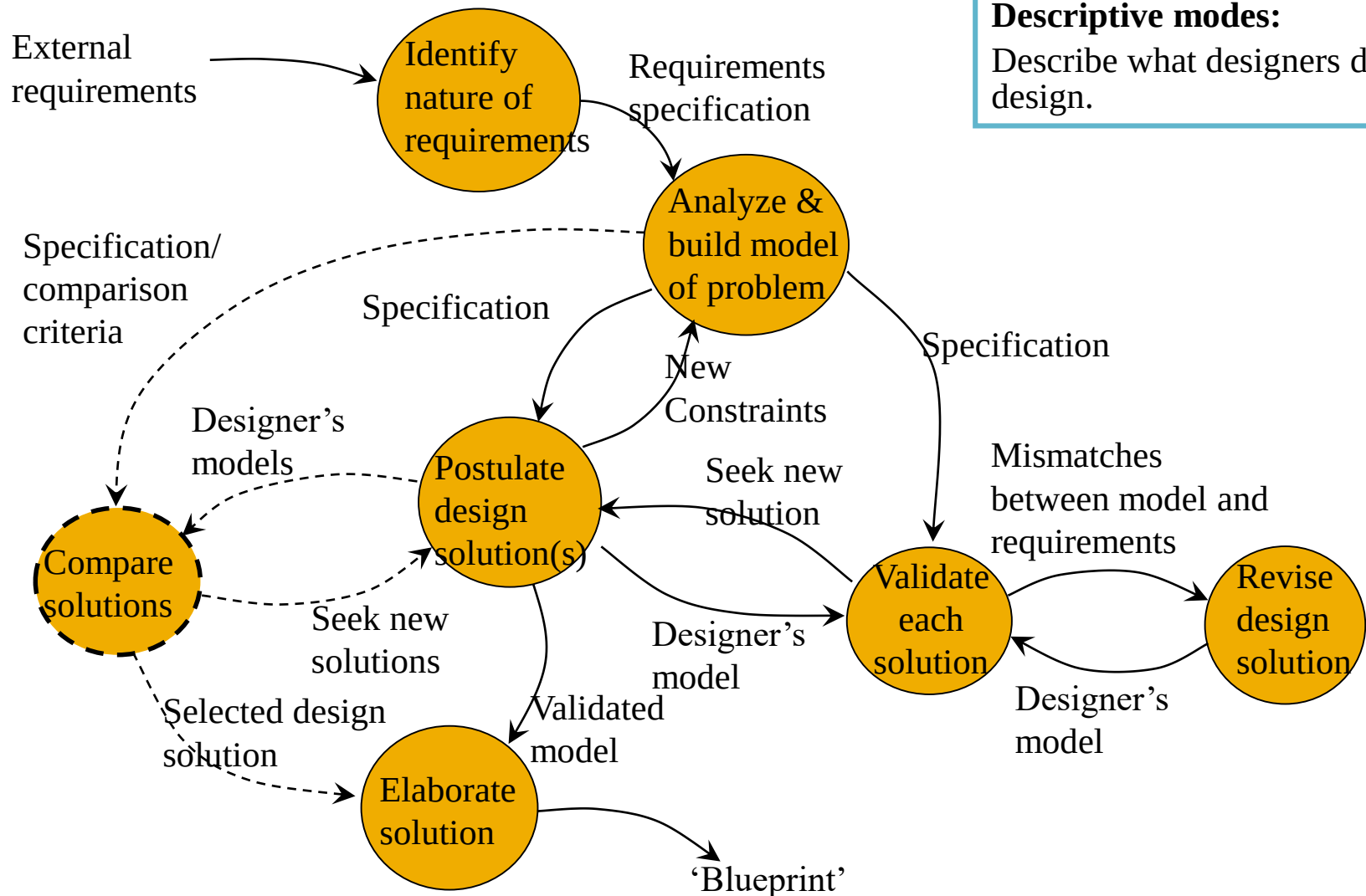
- Components are independent
  - Message received?
  - Solution: complicated protocols
- Correctness is hard to achieve



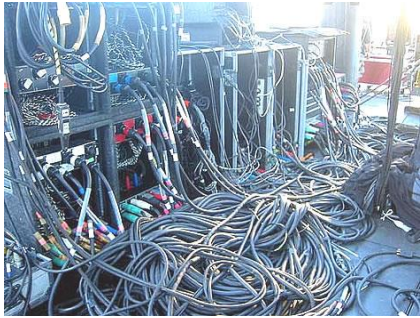
# A Generic Process Model of Designs

## Descriptive modes:

Describe what designers do in design.



# Example of Evolution



➡  
Maintainability



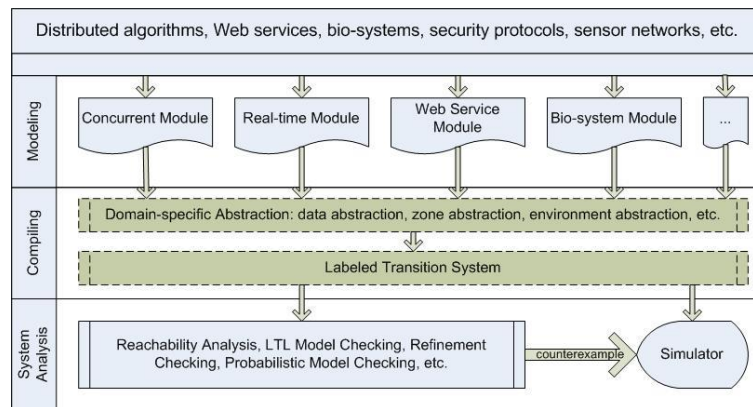
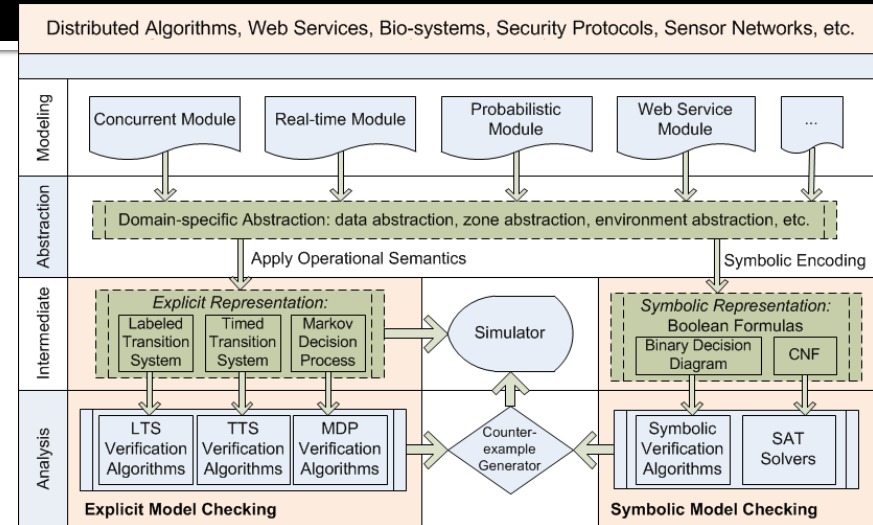
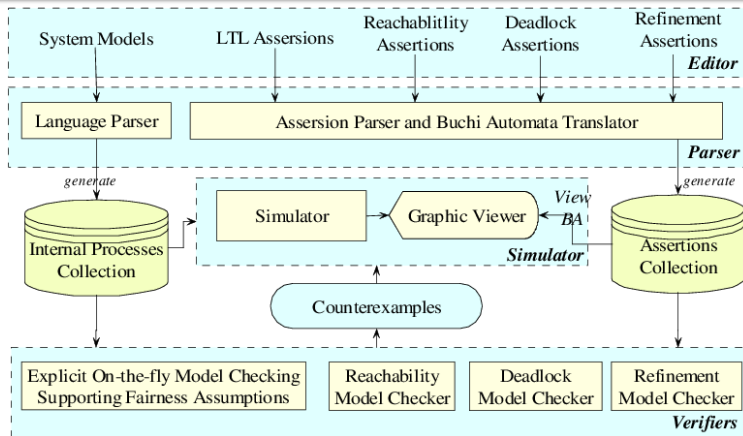
New Features  
➡  
Extensibility



New Features  
⬇  
Performance Scalability



# Real Architecture Diagrams



# Summary of System Design

Why

What

How

